Knowledge Distillation and Production Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

CS3081: Cloud Computing Mini Project Final Report

Author: Mohit Gora | Roll No: 230101056 | GitHub Repository: https://github.com/mohitgora950/cloud | Tag: final

Abstract—This report presents the end-to-end design, implementation, and empirical evaluation of a cloud-native command-parsing microservice. Starting from 3,000 unlabelled natural language commands from the SLURP benchmark, we establish a parallel sharded pipeline backed by SeaweedFS S3 object storage to harvest structured annotations from a teacher model. We train a lightweight 70M-parameter student model (EleutherAI/pythia-70m) using sequence-level knowledge distillation on CPU, achieving 97.0% JSON schema validity and 34.0% full-frame parsing accuracy. The inference engine is containerized strictly under 2.0 GB (1.70 GB disk / 360 MB content) using PyTorch 2.6.0 CPU-only wheels, deployed onto a local Kubernetes (k3s) cluster with Traefik ingress, Horizontal Pod Autoscaling (HPA), and zero-downtime rolling updates. We mathematically partition a 10.20.0.0/16 address block into four subnets, enforce security via a verified stateless Network ACL, and automate testing with GitHub Actions CI/CD. Comprehensive load testing, fault recovery, and SRE cost modeling validate system resilience and production readiness.

1. Task 1: Object Storage & Parallel Teacher Labelling

Architecture & System Decisions: SeaweedFS was deployed in the data namespace to provide S3-compliant object storage with low memory overhead, backed by local persistent volumes. Two separate buckets were provisioned: datasets (raw commands pool and sanitized splits) and models (versioned weights and tokenizers). To label 3,000 natural language utterances without cross-worker race conditions or distributed file locks, we adopted a deterministic modulo-sharding strategy across parallel worker pods.

Implementation & Modulo Sharding: Two worker pods (SHARD_COUNT=2) were deployed in the pipeline namespace with SHARD_INDEX=0 and SHARD_INDEX=1. Each worker independently streams commands_pool.jsonl, filters commands where line_idx % SHARD_COUNT == SHARD_INDEX, queries the teacher model (Ollama / Qwen2.5-7B) using the strict four-key JSON prompt in prompts/teacher_prompt.txt, and writes its partition directly to datasets/teacher/shard-{INDEX}.jsonl. Because each line number modulo K has a unique remainder, the shards form an exact partition with zero overlap and zero gaps (S0 intersect S1 is empty, S0 union S1 is S).

Sanitization & Empirical Results: The sanitization script worker/clean.py reconciled all 3,000 raw teacher outputs against data/schema.json. A total of 223 records were rejected across five cascading filters, yielding exactly 2,777 pristine training records.

Filtering Stage / Condition	Criteria	Count	SHA-256 Checksum
Total Input Records	Raw commands from pool	**3,000**	N/A
(a) Malformed JSON	JSONDecodeError on teacher output	17	N/A
(b) Invalid Schema Keys	Keys != {intent, date, time, place}	14	N/A
(c) Unsupported Intent	Intent not in 8 permitted intents	17	N/A
(d) Hallucinated Slots	Slot text not exact substring of input	128	N/A
(e) Duplicate Commands	Identical text (kept lowest id)	47	N/A
Cleaned Output (v1/train.jsonl)	**Sanitized, sorted, canonical records**	**2,777**	**70393bdcd3b7a2710152e46cfa11...7af**

Limitations: SeaweedFS runs as a standalone deployment without inter-node replication; hardware failure on the host node destroys ephemeral bucket storage. **Improvements:** Deploy a replicated multi-master SeaweedFS statefulset with automated S3 backing store sync.

2. Task 2: Student Training & Sequence-Level Distillation

Distillation Theory & Formulation: In standard *logit-based knowledge distillation* (Hinton et al.), the student is trained by minimizing the Kullback-Leibler (KL) divergence between student logits and teacher probability distributions over the entire vocabulary. However, **logit-based distillation is fundamentally impossible in this setup** because the teacher model is an external black-box LLM accessed strictly via an inference API that outputs generated text strings rather than soft probability distributions. Furthermore, the teacher and student possess entirely disjoint vocabularies and tokenizer architectures (Qwen2.5 vs GPT-NeoX), rendering direct logit alignment mathematically undefined. We therefore employ *sequence-level distillation* (Kim & Rush), where teacher text outputs serve as hard pseudo-ground-truth targets for autoregressive language modeling: L_seq = -sum(log P_S(y_t | y_<t, X)).

Training Configuration & Pipeline: Fine-tuning was executed on the first 1,500 records of train.jsonl on CPU using Hugging Face Transformers and PyTorch. Parameters were pinned in training/config.yaml: seed=42, learning_rate=0.0001, batch_size=16, epochs=3, max_seq_len=56, AdamW optimizer with cosine learning rate schedule. Cross-entropy loss steadily decreased across epochs: Epoch 1 = 2.8566, Epoch 2 = 1.9421, Epoch 3 = 1.2184, recorded in training/loss.csv. Both Model v1 (1 epoch) and Model v2 (3 epochs) were saved with SHA-256 checksums to S3 bucket models and GitHub Releases.

Model Architecture / Checkpoint	JSON Validity	Intent Acc.	Date Acc.	Time Acc.	Place Acc.	Full-Frame
Teacher (Qwen2.5-7B Instruct)	100.0%	85.0%	89.0%	86.0%	83.0%	**46.0%**
Untrained Pythia-70M (Base)	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Student v1 (1 Epoch)	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Student v2 (3 Epochs, Distilled)	**97.0%**	**77.0%**	**81.0%**	**77.0%**	**74.0%**	**34.0%**

Empirical Assessment: Student v2 achieved 97.0% JSON validity, easily beating the >= 95% quality gate requirement. Its full-frame accuracy of 34.0% significantly surpasses the rubric threshold of half the teacher's accuracy (46.0% * 0.5 = 23.0%). **Limitations:** Teacher hallucinations directly poison student supervision. **Improvements:** Implement multi-teacher majority voting and direct preference optimization (DPO).

3. Task 3: Inference Service Containerization & Resource Limits

Service Design: Built using FastAPI and Uvicorn in service/main.py, exposing POST /parse, GET /healthz, and GET /readyz. All configuration is injected via environment variables: MODEL_URI, MODEL_VERSION, S3_ENDPOINT, and MAX_NEW_TOKENS. AWS S3 credentials are strictly loaded via a Kubernetes Secret (s3-credentials). Every response sets custom headers: X-Model-Version and X-Served-By (the Pod hostname). When FAULT_INJECT=1, the service replaces model output with invalid syntax, triggering a strict HTTP 500 with {"error": "invalid_model_output"}.

Container Optimization: Default PyTorch wheels exceed 4.5 GB due to bundled CUDA/cuDNN libraries. We engineered service/Dockerfile to pull official CPU-only wheels from https://download.pytorch.org/whl/cpu (wheel size: 178.6 MB). The final container image has a disk footprint of 1.70 GB (360 MB content), strictly complying with the 2.0 GB limit.

Resource Stress Test: The container was benchmarked under strict cgroup limits (1 vCPU, 1 GiB RAM). Over 50 sequential requests, memory peaked at 457.2 MiB (44.6% of limit) with zero OOM events or process restarts (verified in results/container_memory.txt).

4. Task 4: Kubernetes Deployment, High Availability & Reliability

Cluster Architecture: Deployed to a k3d local cluster in the app namespace, comprising a Deployment (2 replicas), ClusterIP Service, Traefik Ingress routing /parse on port 8080, and a Horizontal Pod Autoscaler (HPA: min 1, max 2, target CPU 70%). Containers are bounded by requests (250m CPU, 512Mi RAM) and limits (1000m CPU, 1024Mi RAM). Readiness probes guard /readyz with 10s initial delay to prevent premature routing while S3 weights download.

High Availability & Failover Telemetry:

- *Zero-Downtime Rolling Update:* Added a lifecycle.preStop.exec.command: ["sleep", "3"] hook to allow kube-proxy to drain endpoints before Uvicorn terminates. During an active load test rolling update, 27/27 requests succeeded (0.00% failure rate; results/rolling_update.txt).
- *Rollback Verification:* Rolling back via kubectl rollout undo to Revision 2 successfully restored Model v1, verified by X-Model-Version: v1 in HTTP headers (results/rollback_v1.txt).
- *Pod Failure Recovery:* Deleting an active pod triggered automated replica replenishment within 22.95s, maintaining 100% availability (0.00% request loss across 26 requests; results/pod_recovery.txt).

- *Cold Start:* Three independent cold-start measurements from pod creation to Ready: Run 1 = 22.03s, Run 2 = 22.35s, Run 3 = 20.88s; **Median = 22.03s** (results/cold_start.txt).
- *Load Balancing:* Under 100 requests through Ingress, traffic split evenly across replicas (37% and 38%), exceeding the >=30% requirement.

Limitations: Single-node laptop execution cannot simulate physical rack failures or split-brain network partitioning.
Improvements: Introduce multi-node k3d clusters with topology spread constraints and PodDisruptionBudgets.

5. Task 5: Subnetting & Stateless Network Access Control

Subnet Partitioning: The assigned block 10.20.0.0/16 (65,536 addresses) was divided into four contiguous subnets using the minimal prefix length satisfying 2^(32-prefix) - 2 >= Hosts Required.

Subnet Name	Required	Network Address / Prefix	Usable Host Range	Broadcast Address	Usable Capacity
app	1,000	**10.20.0.0/22**	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022 hosts
public	250	**10.20.4.0/24**	10.20.4.1 – 10.20.4.254	10.20.4.255	254 hosts
data	100	**10.20.5.0/25**	10.20.5.1 – 10.20.5.126	10.20.5.127	126 hosts
management	20	**10.20.5.128/27**	10.20.5.129 – 10.20.5.158	10.20.5.159	30 hosts

Stateless NACL Dynamics: In network/nacl.json, the public subnet NACL isolates the public load balancer. Because NACLs are strictly stateless, they do not maintain connection tracking tables; reverse reply packets must have explicit permit rules matching ephemeral ports 1024–65535. Furthermore, NACL rules evaluate in ascending numeric order on a first-match basis. Therefore, Rule 100 explicitly denies all inbound traffic from abusive CIDR 198.51.100.0/24 *before* Rules 200/210 permit web traffic (ports 80/443). Simulated against network/packets_sample.jsonl, our rules achieved a 100.0% verification score (6/6).

6. Task 6: Reproducibility, CI/CD & Security Automation

Continuous Integration Pipeline: Configured in .github/workflows/ci.yaml with four sequential jobs: (1) Secret Scanning with Gitleaks (reports zero leaks), (2) Flake8 Python linting, (3) PyTest unit testing (health, schema validation, fault injection, parse response), and (4) Release Model Evaluation & Quality Gate. The quality gate executes tools/score.py --min-validity 95. When an untrained model is tested, the gate fails immediately, blocking bad deployments. **One-Command Deployment:** Makefile automates the entire lifecycle: make up bootstraps the cluster, installs SeaweedFS, builds the image, and deploys manifests in 3 automated steps. make down safely tears down all cluster resources.

7. Task 7: Performance, Cost & SRE Review

Locust Load Test Empirical Telemetry: A 2-minute headless Locust benchmark was executed with 4 unthrottled concurrent users (spawn rate 4) against the Ingress endpoint (http://localhost:8080/parse). Telemetry was captured in results/loadtest_stats.csv and results/pod_resources.txt:

- *Latency & Throughput:* Median latency (p50) = **2,500 ms**, min latency = 1,118 ms, average latency = 5,664 ms, p95 latency = 30,000 ms. Measured throughput = **0.68 req/s** (71 successful requests served in 120s).
- *Error Rate:* Under extreme unthrottled 4-user concurrency on CPU, 8 requests exceeded the 30s Locust timeout (error rate = 10.13%).
- *Pod Resource Utilization:* Pod memory remained highly stable between **338 MiB and 350 MiB** (peak 34.2% of 1 GiB limit). Under load, aggregate CPU rose from 3m idle to 3,037m across the 4 active replicas.
- *Cold-Start Latency:* Median cold start = **22.03s**.

Cost Estimation (Rate Card Formula): Using data/rate_card.json, hourly rates are: vCPU hour = 3.50 INR, GiB memory hour = 0.45 INR. Each replica limit is 1 vCPU and 1 GiB memory = 3.50 + 0.45 = 3.95 INR/hour.

Hourly cost of N replicas = N * 3.95 INR.
For 4 running replicas: Hourly Cost = 4 * 3.95 = **15.80 INR/hour**. (For 2 replicas: 7.90 INR/hour).
At measured throughput of 0.68 RPS: Requests per hour = 0.68 * 3,600 = 2,448 req/hour.
Cost per 1,000 requests = (15.80 / 2,448) * 1000 = 6.454 INR (or 7.418 INR based on 2,130 successful requests/hour).

SLO & Error Budget Analysis:
- *SLI / SLO Target:* 99.0% of requests return HTTP 200 within 5,000 ms.
- *Window:* One load-test execution window (79 total requests over 120 seconds).
- *Error Budget:* Permitted errors = 79 * (1 - 0.99) = **0.79 requests** (effectively zero errors permitted).

• *Budget Consumption:* 8 requests timed out due to host CPU saturation, consuming 8 / 0.79 = 1,012.6% of the error budget. Production mitigation requires rate limiting (HTTP 429) and GPU auto-scaling.

Comprehensive SRE Review Table:

Component / Concern	Evidence (Measured Value)	Observed Limitation	Proposed Architectural Improvement
Data and storage	2,777 records kept (SHA-256: 70393bdcd...); 223 rejected (17 JSON, 14 keys, 17 intent, 128 slots, 47 dupes).	SeaweedFS deployed as single-replica pod with local volume; lack of replication risks data loss.	Deploy SeaweedFS in distributed replication topology (replication '001') with S3 tiering.
Model quality	Student v2: 97.0% JSON validity, 77.0% intent, 81.0% date, 34.0% full-frame (untrained scored 0.0%).	70M parameter capacity is prone to repetition loops on unseen out-of-distribution entities.	Implement grammar-guided constrained decoding (e.g., Outlines/Jsonformer) to guarantee 100% schema compliance.
Latency and throughput	p50 = 2,500 ms, min = 1,118 ms, avg = 5,664 ms, throughput = 0.68 req/s under 4 concurrent CPU users.	Autoregressive token generation on CPU without GPU acceleration causes latency to scale linearly (~40ms/tok).	Export Pythia-70M to INT8/INT4 via ONNX Runtime / OpenVINO, reducing latency by 3–4x.
Resource utilisation	Memory usage: 338–350 MiB per pod (34.2% of 1 GiB limit); CPU peaked at 3,037m across replicas.	CPU contention during multi-user spikes leads to queue buildup and thread thrashing.	Enforce CPU core pinning (cpuset) and set PyTorch intra-op threads (torch.set_num_threads(2)).
Reliability and recovery	Rolling update: 0.00% drops (27/27); pod recovery: 22.95s (0.00% drops); median cold start: 22.03s.	Ingress depends on single Traefik controller; single-node failure crashes the routing plane.	Deploy multi-replica Traefik with PodDisruptionBudget (PDB) and pod anti-affinity rules.
Security and cost	Stateless NACL: 6/6 test packets passed (100.0%); 7.42 INR per 1,000 requests (15.80 INR/hr for 4 pods).	S3 credentials in secret.yaml are base64 strings in git; pods run as default root user.	Integrate external-secrets-operator with HashiCorp Vault; enforce non-root securityContext.

8. Conclusion & Submission Checklist

All project objectives were completed and empirically verified on the cloud-cluster environment: clean S3 data sharding (Task 1), successful Pythia-70M sequence-level distillation passing the 95% quality gate (Task 2), 1.70 GB optimized Docker container with memory stress verification (Task 3), high-availability Kubernetes deployment with zero-downtime rolling updates and rollback (Task 4), mathematical subnetting and stateless NACL design (Task 5), reproducible CI/CD workflow with Gitleaks and Makefile automation (Task 6), and empirical Locust performance, cost, and SRE review (Task 7). Source code is tagged as final in the repository, and deliverables are archived strictly per assignment standards.